

Data Mining

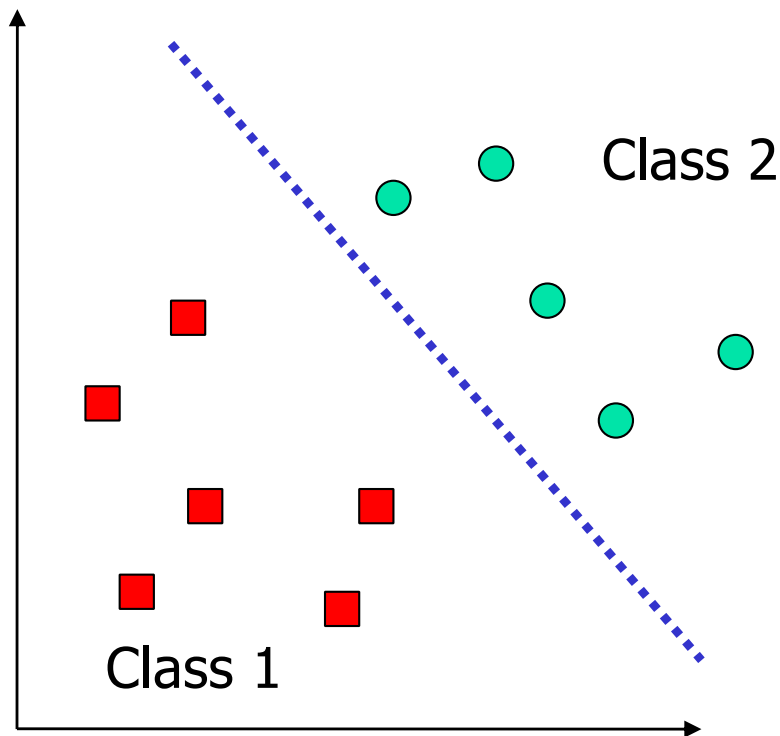


Support Vector Machine (SVM) Maximum Margin Classifier & Kernel Functions

- Supervised Learning Method
- Used for:
 - Classification
 - Regression
- Known for:
 - Find the best decision boundary

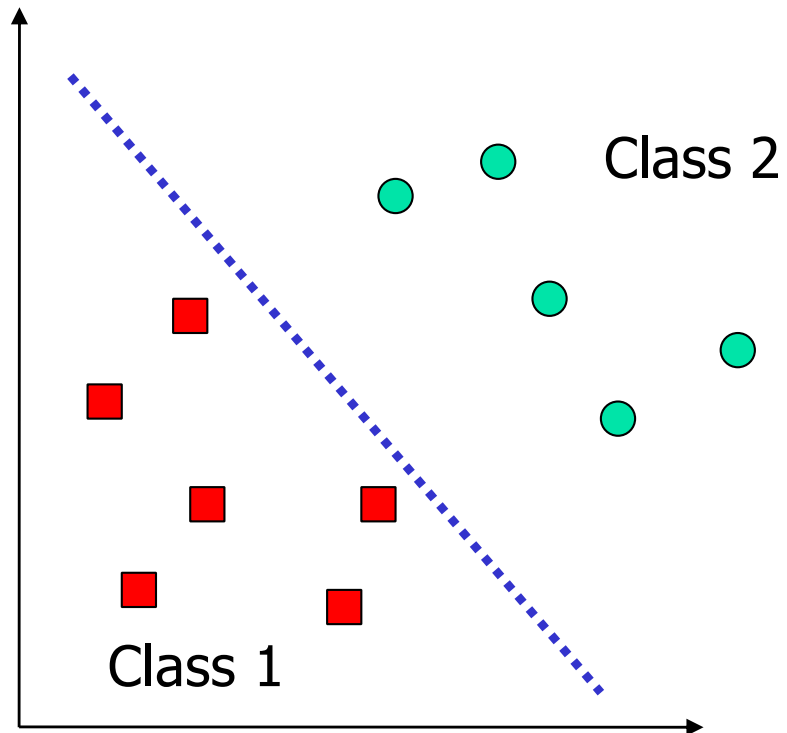
Two Class Problem: Linear Separable Case

Many decision boundaries can separate these two classes
Which one should we choose?

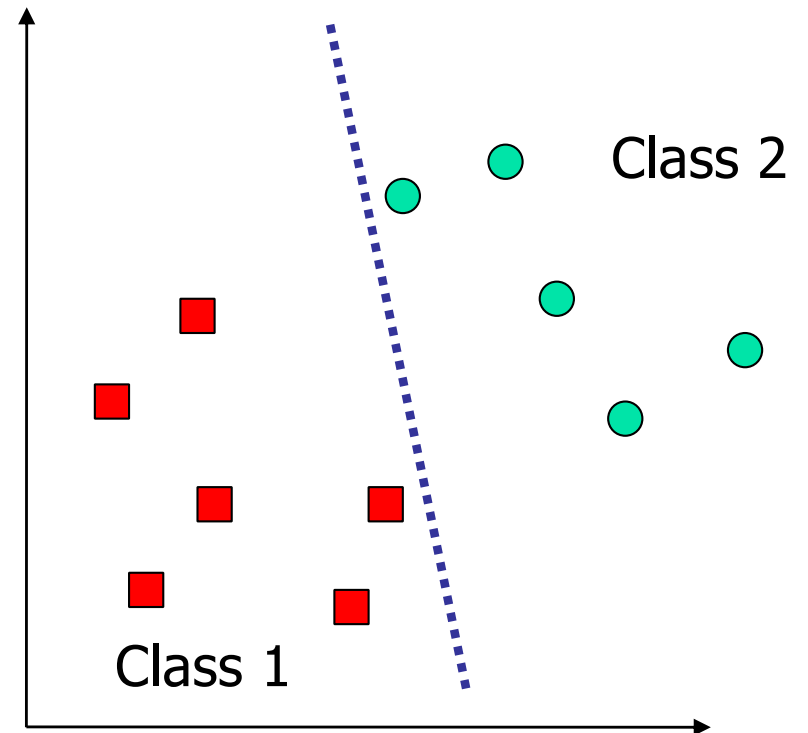


Example of Bad Decision Boundaries

Too close to data $\rightarrow$ unstable



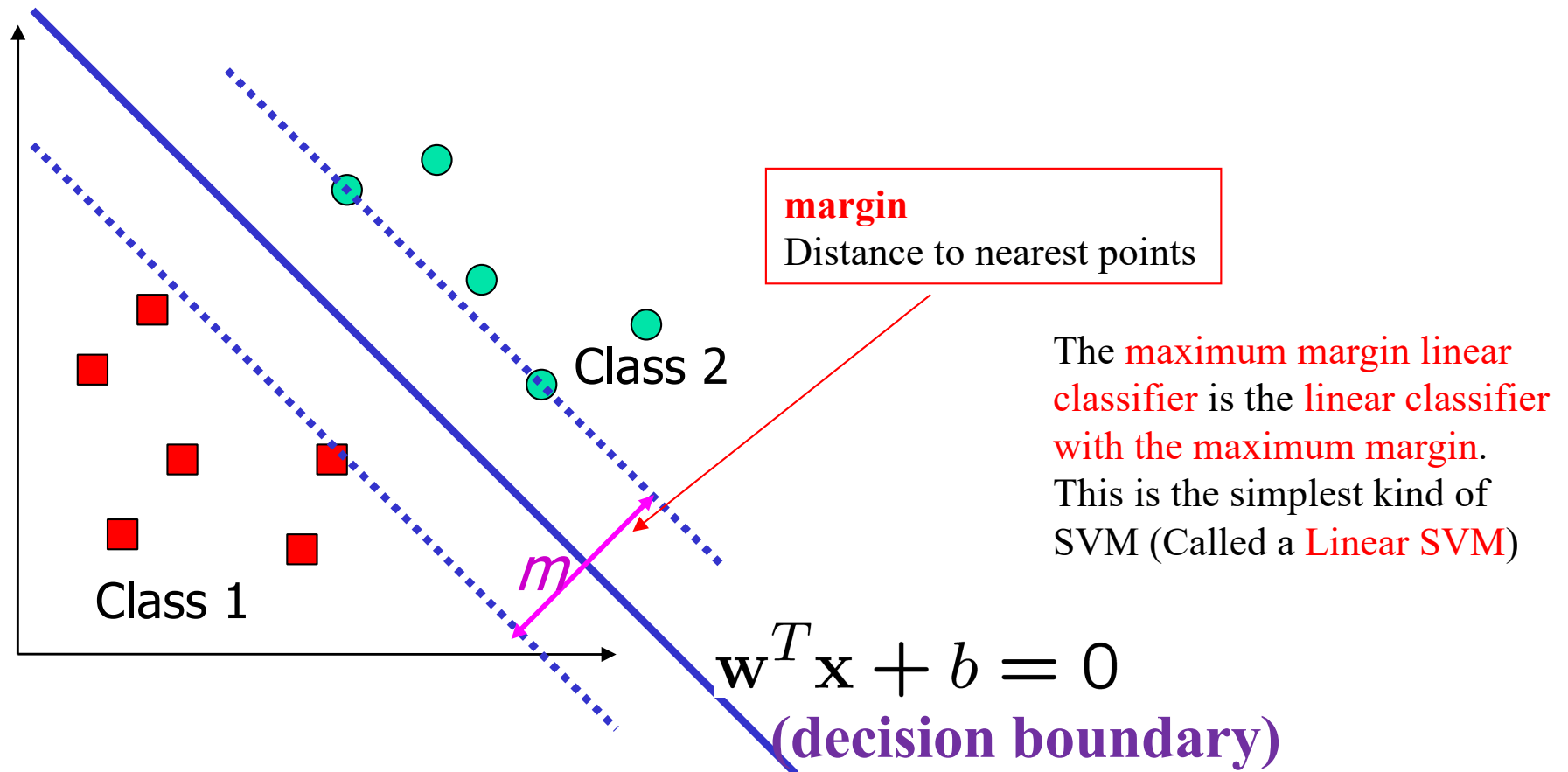
Sensitive to noise



Good Decision Boundary: Margin Should Be Large

The **decision boundary** should be as far away from the data of both classes as possible

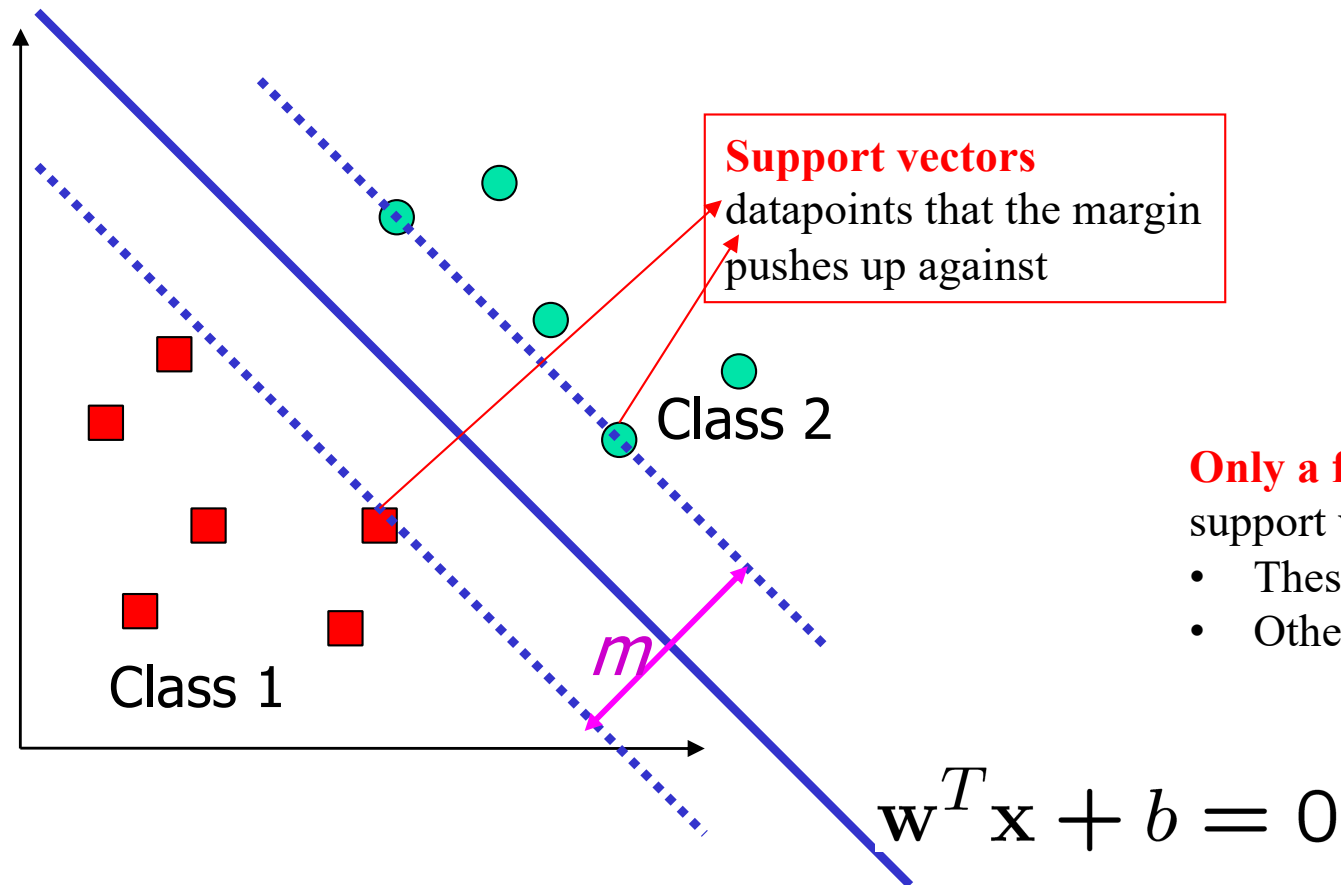
- We should **maximize** the **margin**, m



Good Decision Boundary: Margin Should Be Large

The decision boundary should be **as far away from the data of both classes as possible**

– We should **maximize** the **margin**, m



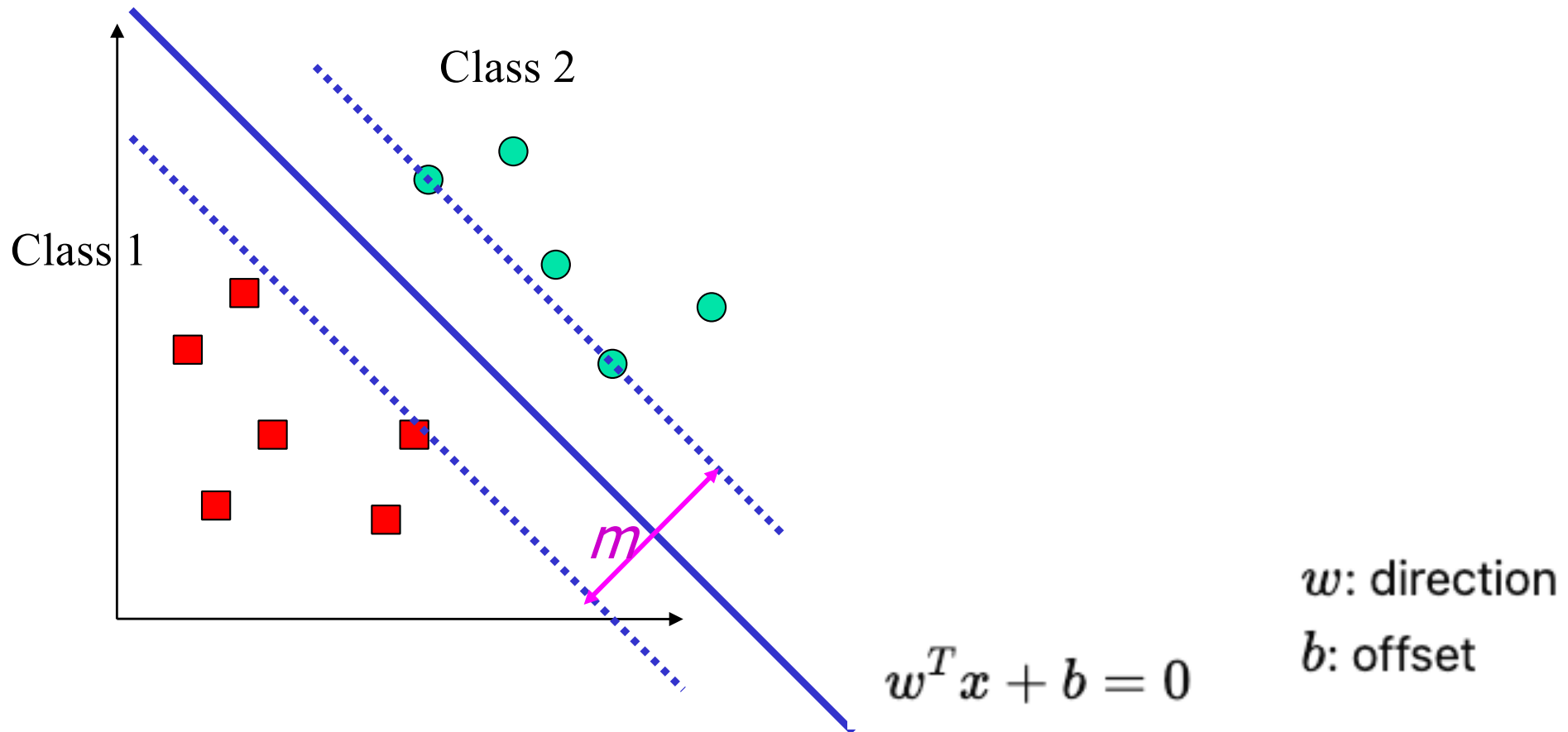
Only a few points matter! (The support vectors)

- These points define the boundary
- Others do NOT affect the model

What is the value of length of m ?

The decision boundary should be as far away from the data of both classes as possible

- We should maximize the margin, m



Compute the length of m

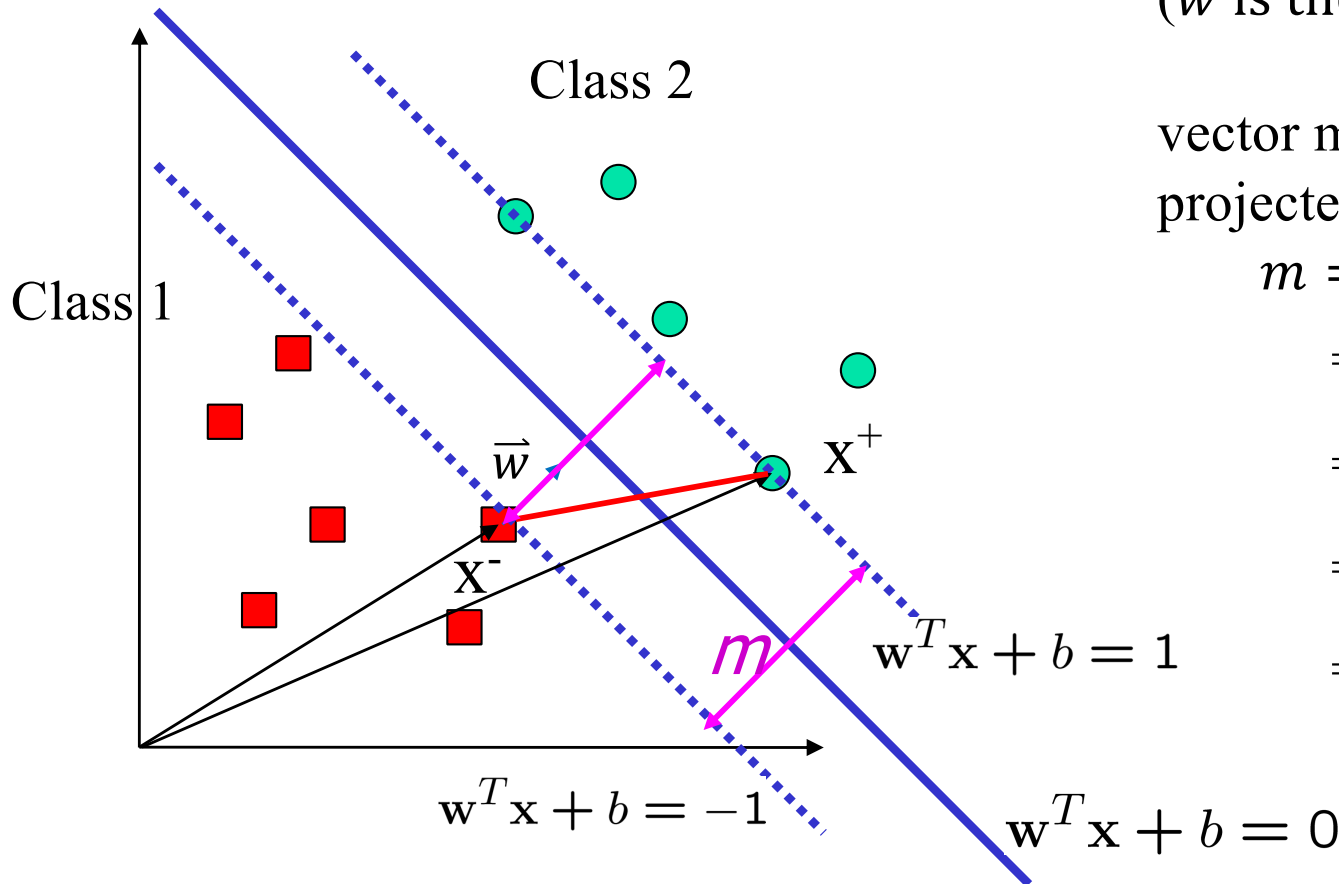
The decision boundary should be **as far away from the data of both classes as possible**

– We should **maximize** the **margin**, $\|m\|$

vector w perpendicular to the decision boundary $w^T x + b = 0$
($\hat{w}$ is the unit vector of w)

vector m is the vector $\vec{u} = x^+ - x^-$ projected on vector w :

$$\begin{aligned} m &= \vec{u} \cdot \hat{w} \\ &= (x^+ - x^-) \cdot \frac{\vec{w}}{\|w\|} \\ &= (x^+ \cdot \vec{w} - x^- \cdot \vec{w}) \cdot \frac{1}{\|w\|} \\ &= ((1-b) - (-1-b)) \cdot \frac{1}{\|w\|} \\ &= \frac{2}{\|w\|} \end{aligned}$$



Margin Formula

$$\text{margin} = \frac{2}{\|w\|}$$

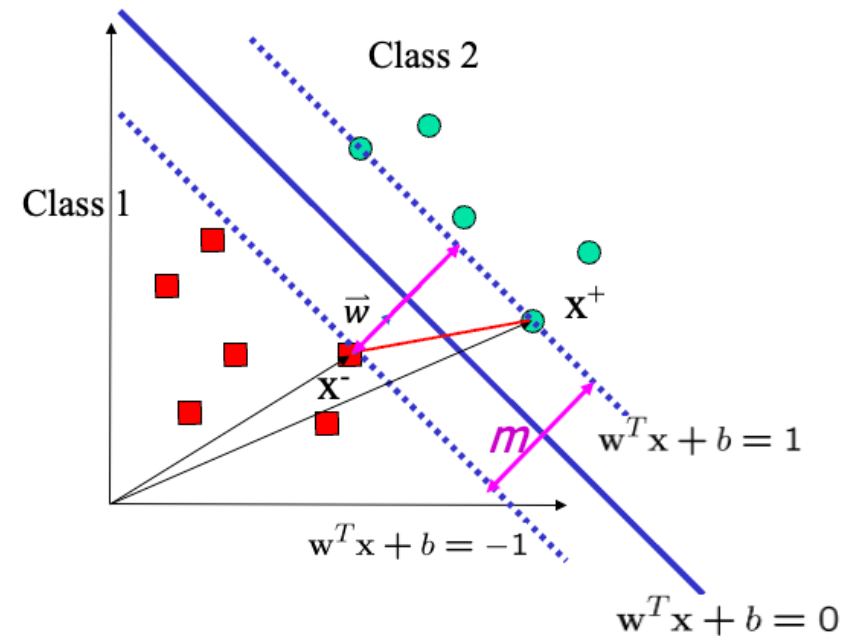
Smaller $\|w\| \rightarrow$ larger margin

For correct classification:

$$y_i(w^T x_i + b) \geq 1$$

- correct side
- The minimum perpendicular distance from any training point to the decision boundary

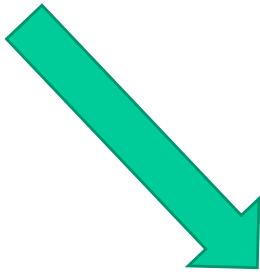
Ensures each point is correctly classified and at least a minimum distance from the boundary



The Optimization Problem

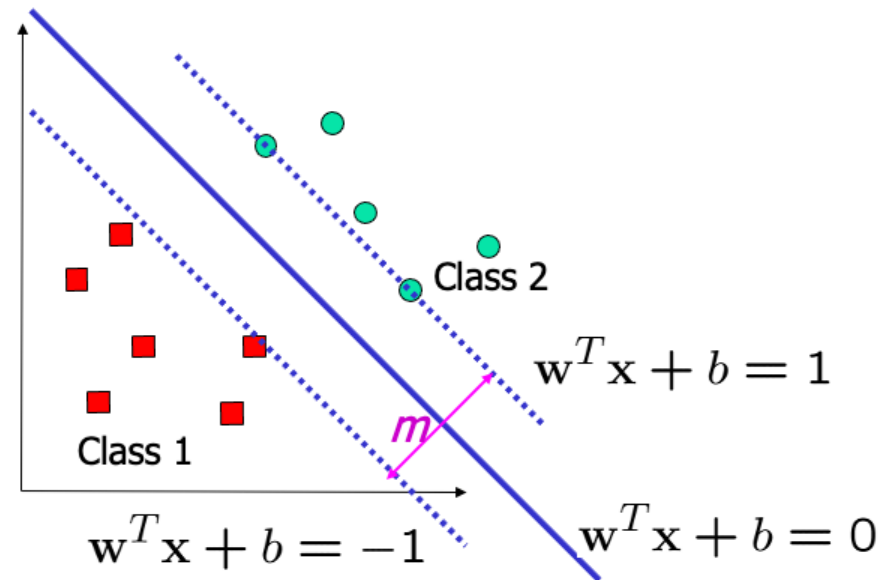
To maximize:

$$m = \frac{2}{\|\mathbf{w}\|}$$



$$\text{Minimize } \frac{1}{2} \|\mathbf{w}\|^2$$

$$\text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i$$



Lagrangian of Original Problem

The Lagrangian is Minimize $\frac{1}{2}||\mathbf{w}'||^2$

subject to $1 - y_i(\mathbf{w}'^T \mathbf{x}_i + b) \leq 0$ for $i = 1, \dots, n$

– Note that $||\mathbf{w}'||^2 = \mathbf{w}'^T \mathbf{w}'$

$$\mathcal{L} = \frac{1}{2} \mathbf{w}'^T \mathbf{w}' + \sum_{i=1}^n \alpha_i (1 - y_i(\mathbf{w}'^T \mathbf{x}_i + b))$$

→ Lagrangian multipliers

Setting the gradient of $\mathcal{L}$ w.r.t. $\mathbf{w}'$ and b to zero, we have

$$\mathbf{w}' + \sum_{i=1}^n \alpha_i (-y_i) \mathbf{x}_i = 0 \quad \Rightarrow \quad \mathbf{w}' = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

$$\sum_{i=1}^n \alpha_i y_i = 0$$

$$\alpha_i \geq 0$$

Change into the Dual Problem

From optimization, we obtained:

$$w = \sum_{i=1}^n \alpha_i y_i x_i$$

Key insights:

$$\sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha_i \geq 0$$

- **Model depends on training data**
- **Only points with $\alpha_i > 0$ matter (these are support vectors)**

Substitute into objective, we get the dual problem

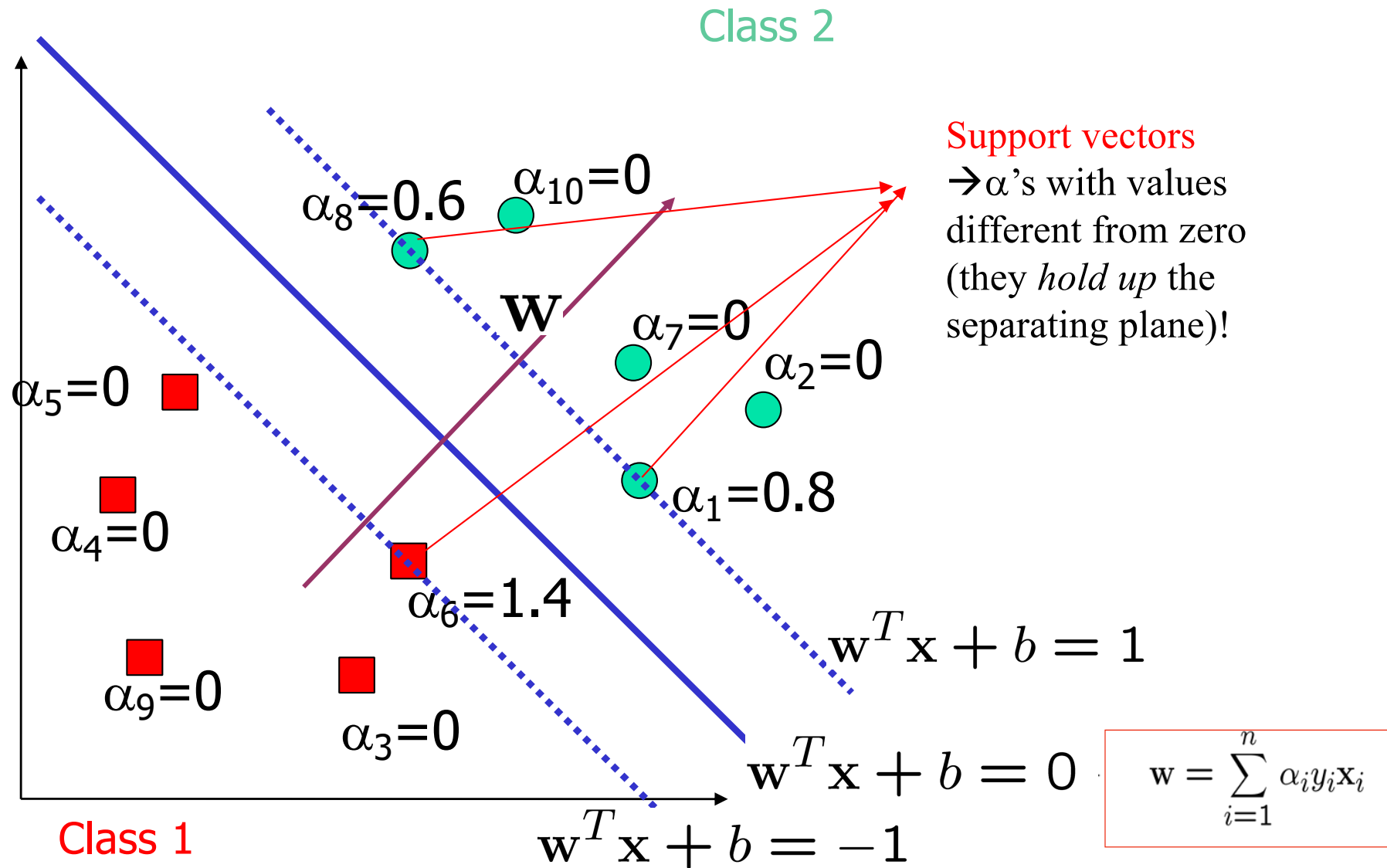
$$\max W(\alpha) = \sum \alpha_i - \frac{1}{2} \sum \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

- Data appears only as: $x_i^T x_j$ (dot products)
- **This enables kernel methods**

This is a convex quadratic programming (QP) problem

- Global maximum of α_i can always be found
- Well established tools for solving this optimization problem

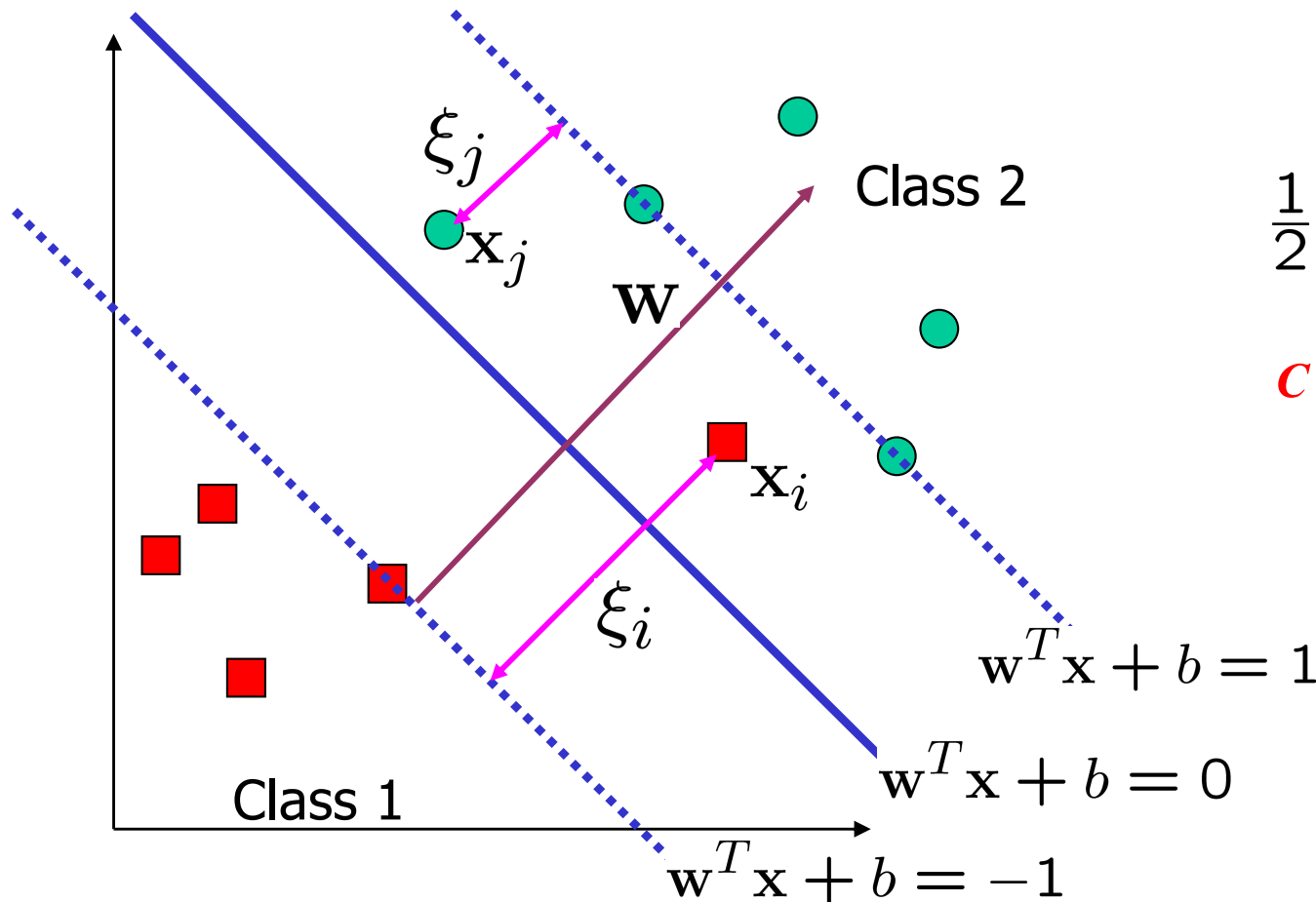
A Geometrical Interpretation



Non-linearly Separable Problems

We allow “error” ξ_i in classification; it is based on the output of the discriminant function $\mathbf{w}^T \mathbf{x} + b$

ξ_i : how much a data point violates the margin (or misclassification)



New objective function:

$$\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

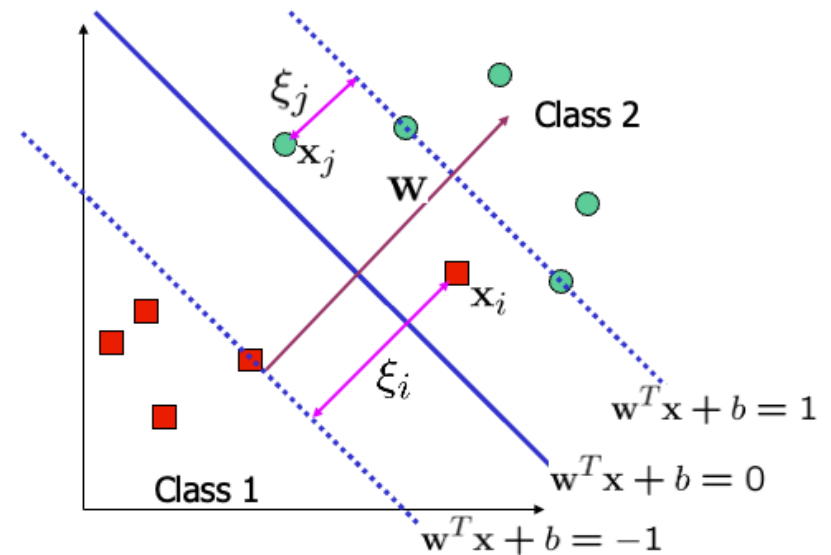
C : tradeoff parameter between error and margin; chosen by the user; large C means a higher penalty to errors

The Updated Objective

Objective Function:

$$\frac{1}{2} \|w\|^2 + C \sum \xi_i$$

- SVM maximizes the margin (by minimizing $\|w\|^2$)
- Penalize violations (via $\sum \xi_i$)



$\xi_i=0$: correct, outside margin

$0 < \xi_i < 1$: correct but inside margin

$\xi_i > 1$: misclassified

$$\text{margin} = \frac{2}{\|w\|}$$

The Optimization Problem

The dual optimization problem is:

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$$

Subject to:

$$0 \leq \alpha_i \leq C, \quad i = 1, \dots, n$$

$$\sum_{i=1}^n \alpha_i y_i = 0$$

Recovering w : $w = \sum_{i=1}^n \alpha_i y_i x_i$

The only difference with the linear separable case is that there is an upper bound C on α_i

Once again, a **QP solver** can be used to find α_i efficiently

Extension to Non-linear SVMs (Kernel Machines)

Non-Linear SVM

How could we generalize this procedure to non-linear data?

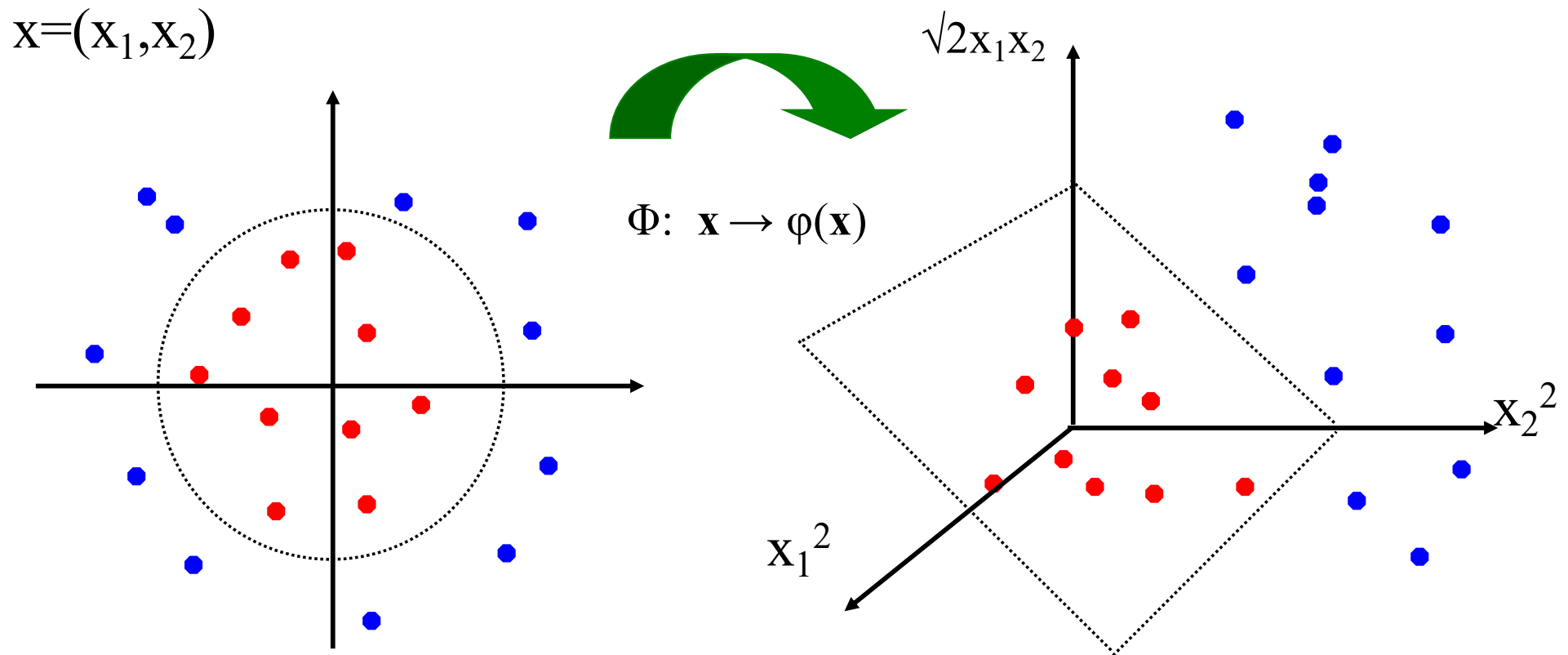
Vapnik in 1992 showed that transforming input data $\mathbf{x}_i$ into a higher dimensional space makes the problem easier. **Similar to Hidden Layers in ANN**

- We know that data appears only as dot products: $\mathbf{x}_i^T \mathbf{x}_j$
- Suppose we transform the data to some (possibly infinite dimensional) space $\mathbf{H}$ via a mapping function Φ such that the data appears of the form $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$

Why?

- Linear operation in $\mathbf{H}$ is equivalent to non-linear operation in input space.

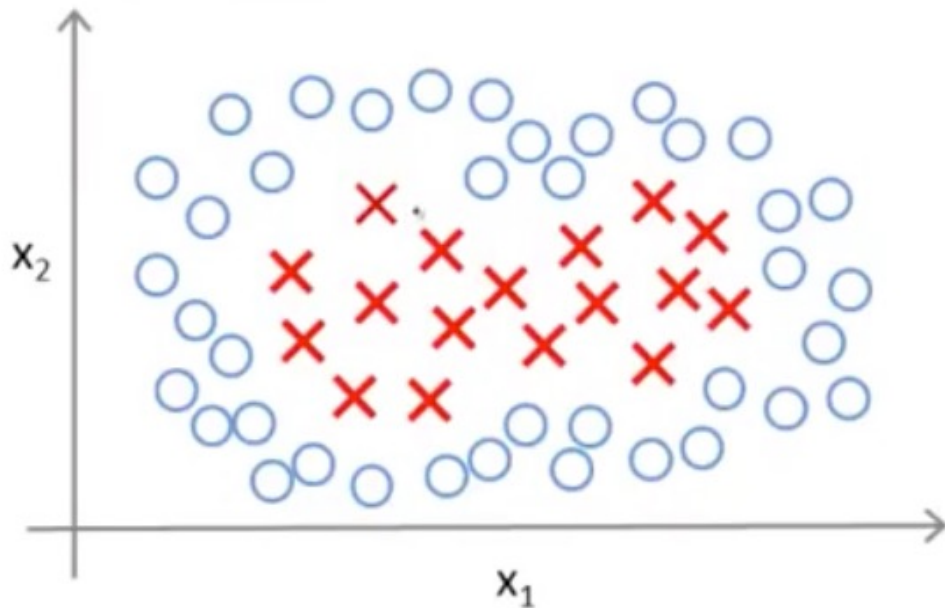
Non-linear SVMs: Feature Space



If data are mapped into a higher space of sufficiently high dimension,
then they will in general be linearly separable;
N data points are in general separable in a space of N-1 dimensions or more!

Non-linear Boundary

Non-linear Decision Boundary



Predict $y = 1$ if

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 + \theta_5 x_2^2 + \dots \geq 0$$

Kernel as Similarity to Landmarks

Given x , construct new features:

$$f_j(x) = K(x, l^{(j)})$$

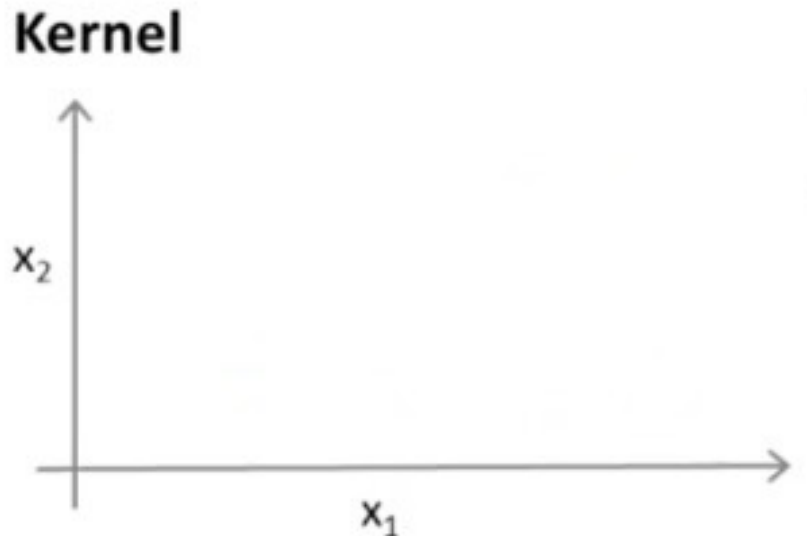
Gaussian Kernel:

$$K(x, l^{(j)}) = \exp\left(-\frac{\|x - l^{(j)}\|^2}{2\sigma^2}\right)$$

$f_j(x)$ measures similarity to landmark $l^{(j)}$

Close $\rightarrow$ large value

Far $\rightarrow$ small value



Gaussian Kernel Intuition

We transform x into similarity-based features:

$$K(x, l) = \exp \left(-\frac{\|x - l\|^2}{2\sigma^2} \right)$$

If $x \approx l$: $\|x - l\|^2 \approx 0 \Rightarrow K(x, l) \approx 1$ High similarity

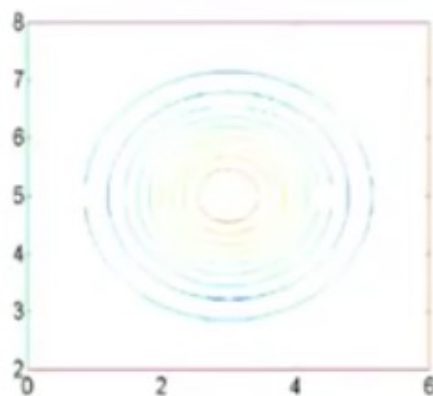
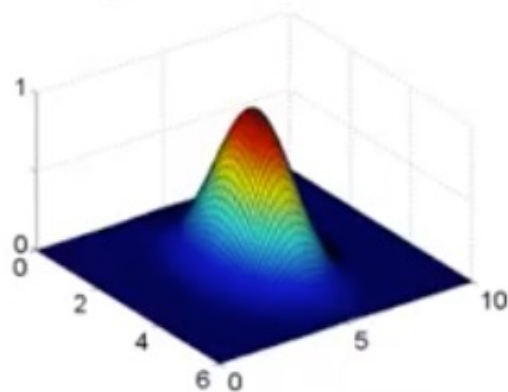
If x is far from l : $\|x - l\|^2$ large $\Rightarrow K(x, l) \approx 0$ Low similarity

Gaussian Kernel

Example:

$$l^{(1)} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}, \quad f_1 = \exp \left(-\frac{\|x - l^{(1)}\|^2}{2\sigma^2} \right)$$

$$\sigma^2 = 1$$

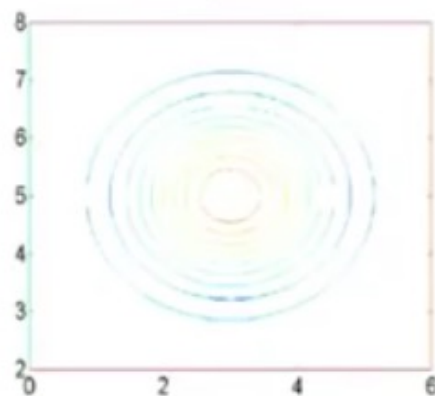
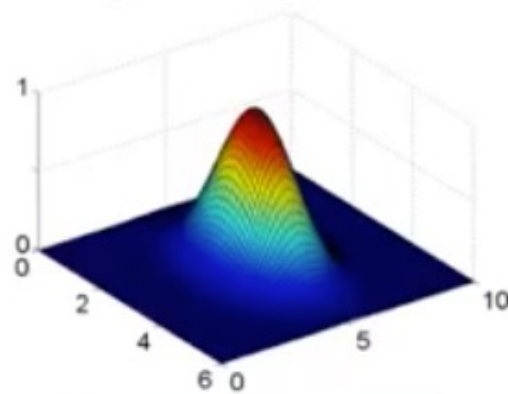


Gaussian Kernel

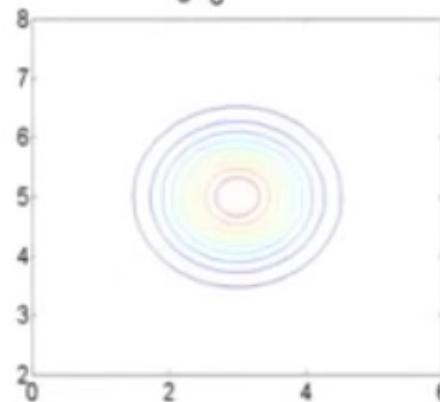
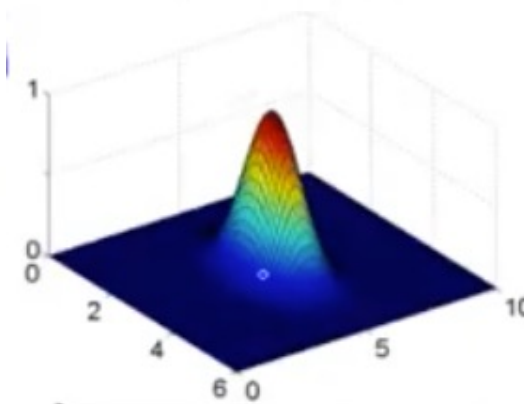
Example:

$$l^{(1)} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}, \quad f_1 = \exp\left(-\frac{\|x - l^{(1)}\|^2}{2\sigma^2}\right)$$

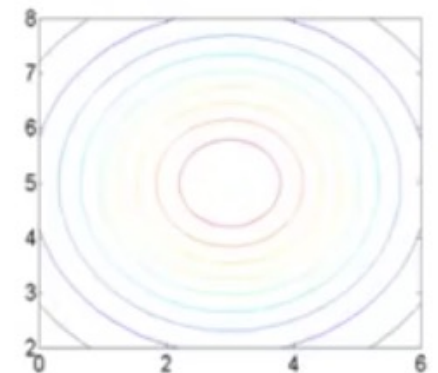
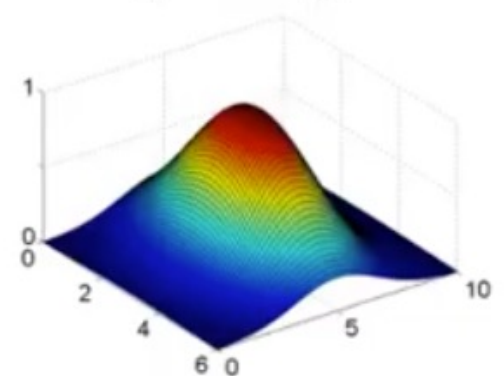
$$\sigma^2 = 1$$



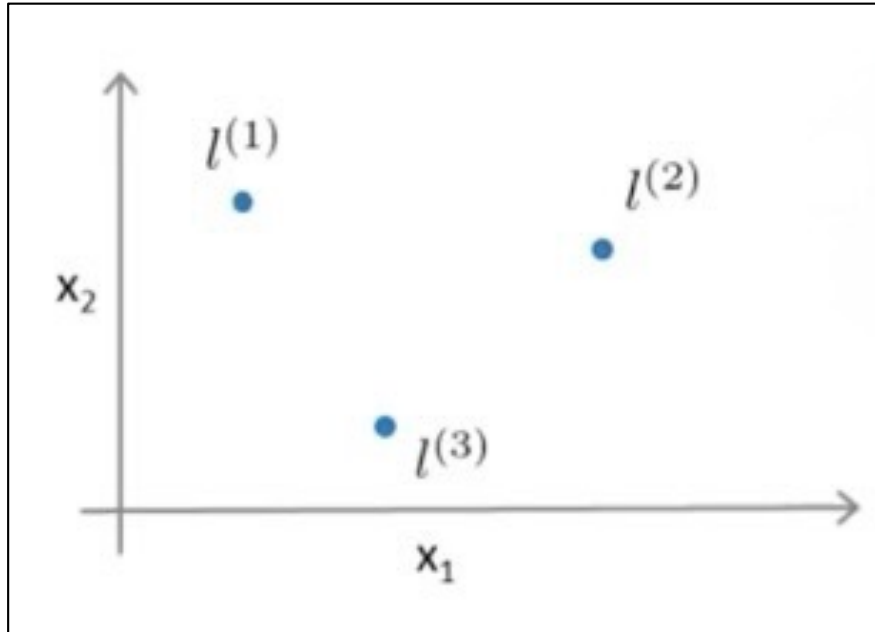
$$\sigma^2 = 0.5$$



$$\sigma^2 = 3$$



Classification with the Kernel



Near a landmark $\rightarrow$ feature value is high
Far from landmarks $\rightarrow$ feature values are small

Define features:

$$f_j(x) = K(x, l^{(j)})$$

Linear model in feature space:

$$h(x) = \theta_0 + \sum_{j=1}^m \theta_j f_j(x)$$

Prediction rule:

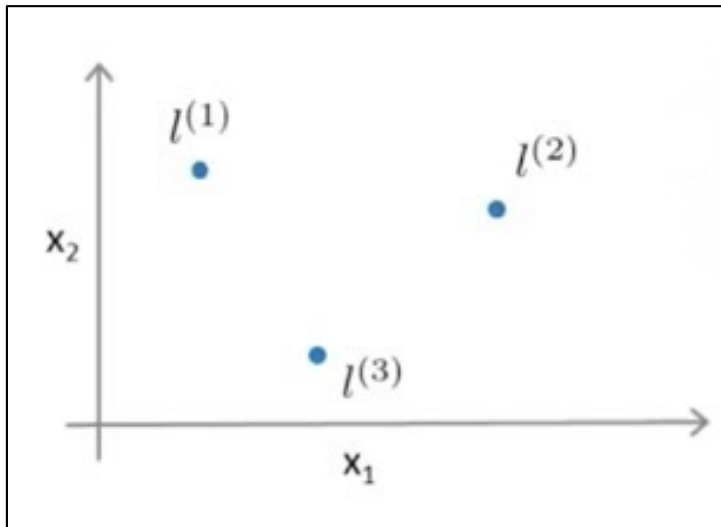
$$\text{Predict } y = 1 \quad \text{if } h(x) \geq 0$$

Nonlinear boundary in original space = linear model in transformed feature space.

θ_0 corresponds to bias b in SVM

Toy Example

$$h(x) = \theta_0 + \sum_{j=1}^m \theta_j f_j(x) \quad \theta_0 = -0.5, \quad \theta_1 = 1, \quad \theta_2 = 1, \quad \theta_3 = 0$$



x near $l^{(1)}$

$$f_1 \approx 1, \quad f_2 \approx 0, \quad f_3 \approx 0$$

$$h(x) \approx -0.5 + 1 = 0.5 \geq 0$$

Predict $y = 1$

x far from all landmarks

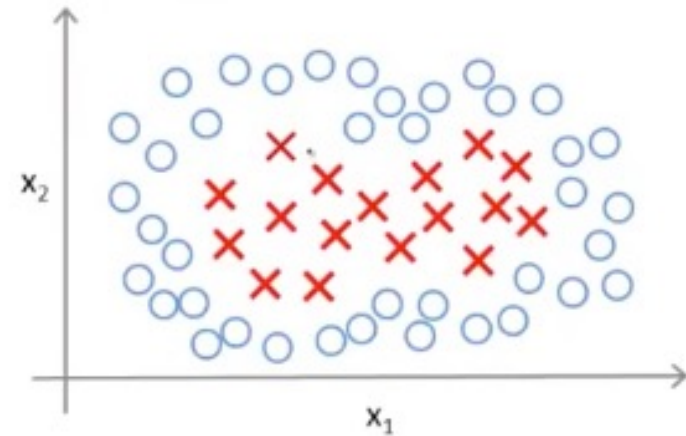
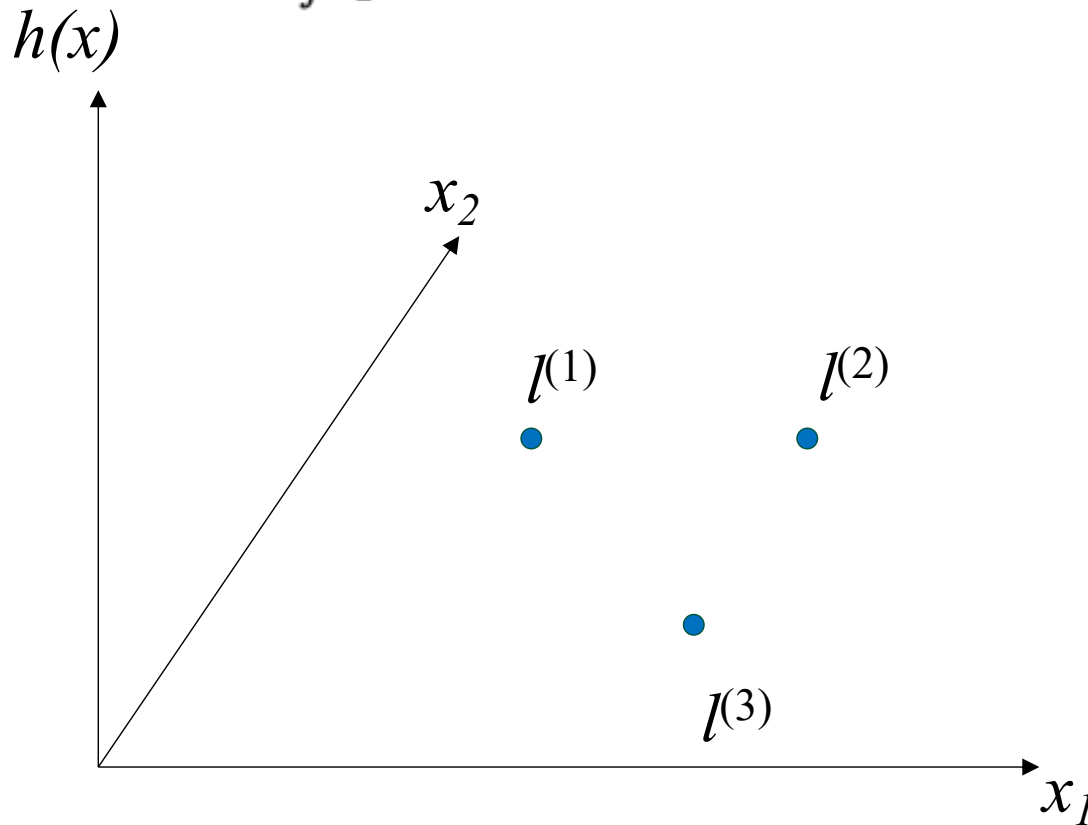
$$f_1 \approx 0, \quad f_2 \approx 0, \quad f_3 \approx 0$$

$$h(x) \approx -0.5 < 0$$

Predict $y = 0$

Toy Example

$$h(x) = \theta_0 + \sum_{j=1}^m \theta_j f_j(x) \quad \theta_0 = -0.5, \quad \theta_1 = 1, \quad \theta_2 = 1, \quad \theta_3 = 0$$



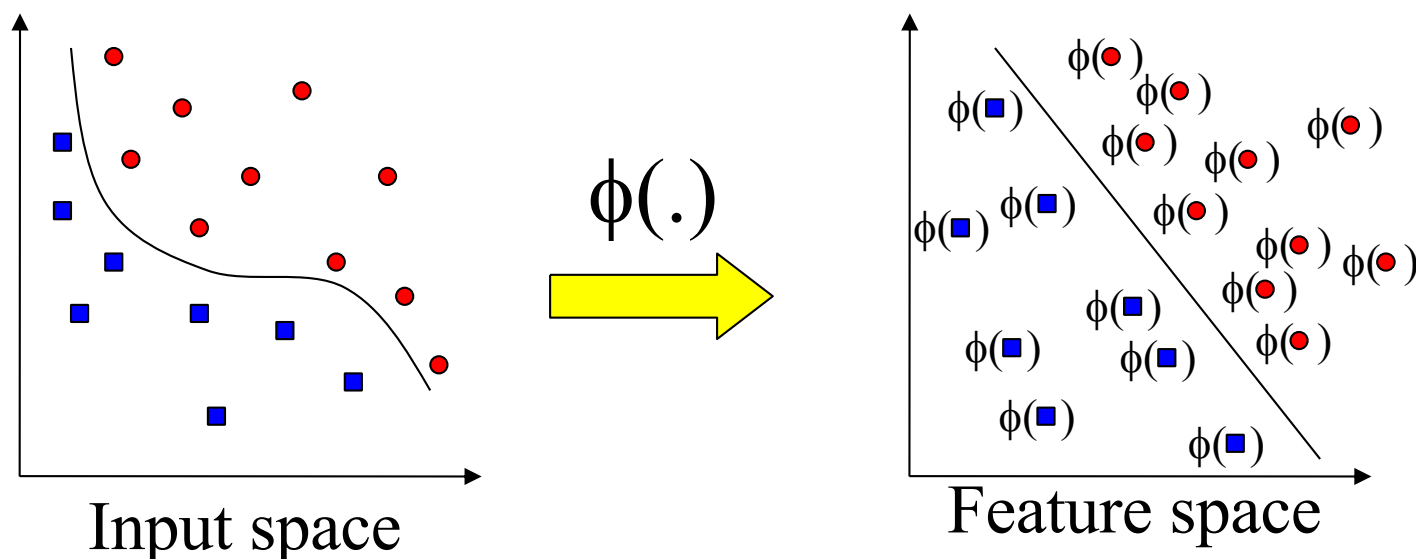
Transformation to Feature Space

Possible problem of the transformation

- High computation burden due to high-dimensionality and hard to get a good estimate

SVM solves these two issues simultaneously

- “Kernel tricks” for efficient computation
- Minimize $\|\mathbf{w}\|^2$ can lead to a “good” classifier



Kernel Trick

Recall:

Note that data only appears as dot products

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \underbrace{x_i^T x_j}_{\text{dot product}}$$
$$C \geq \alpha_i \geq 0, \sum_{i=1}^N \alpha_i y_i = 0$$

Since data is only represented as **dot products**, we **don't need to do the mapping explicitly**.

Introduce a Kernel Function (*) K such that: $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$

Replace: $x_i^T x_j \rightarrow K(x_i, x_j)$

*We can work in **high dimensional space** without explicitly computing $\phi(x)$*

(*)Kernel function – a function that computes inner products in feature space:

$K(x_i, x_j)$ = “similarity” in transformed space

Feature Mapping & Kernel

Feature Mapping:

Let $\mathbf{x} = (x_1, x_2)$, $\mathbf{y} = (y_1, y_2)$

$$\phi(\mathbf{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

$$\phi(\mathbf{y}) = (1, \sqrt{2}y_1, \sqrt{2}y_2, y_1^2, y_2^2, \sqrt{2}y_1y_2)$$

Inner Product in Feature Space

$$\phi(\mathbf{x})^T \phi(\mathbf{y}) = (1 + x_1y_1 + x_2y_2)^2$$

Kernel Function

$$K(x, y) = (1 + \mathbf{x}^T \mathbf{y})^2$$

The inner product $\phi(.)\phi(.)$ can be computed by K **without going through the map $\phi(.)$ explicitly!**

Modification Due to Kernel Function

Original Dual Problem:

$$\begin{aligned} \max_{\alpha} \quad W(\alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to:} \quad &0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Apply Kernel Trick

Replace inner product: $\mathbf{x}_i^T \mathbf{x}_j \rightarrow K(\mathbf{x}_i, \mathbf{x}_j)$

Kernelized Dual Problem:

$$\begin{aligned} \max_{\alpha} \quad W(\alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \\ \text{subject to:} \quad &0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Common Kernel Functions

Polynomial Kernel $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + 1)^d$

- d : degree of the polynomial
- Captures **feature interactions up to degree d**

Gaussian (RBF) Kernel $K(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2}\right)$

- σ : controls width (smoothness)
- Measures **similarity based on distance**
 - Close points $\rightarrow$ value ≈ 1
 - Far points $\rightarrow$ value ≈ 0

Notes:

- Gaussian kernel is closely related to **RBF neural networks**
- Kernel design is an **active research area**

Example: SVM with Polynomial Kernel

Given Data (1D) $x = \{1, 2, 4, 5, 6\}$

Class labels:

- Class +1: 1, 2, 6
- Class -1: 4, 5

$$y = \{+1, +1, -1, -1, +1\}$$

Kernel Choice

- Polynomial kernel (degree 2): $K(x_i, x_j) = (x_i x_j + 1)^2$
- Regularization parameter: $C = 100$

Optimization Problem (Dual)

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^5 \alpha_i - \frac{1}{2} \sum_{i=1}^5 \sum_{j=1}^5 \alpha_i \alpha_j y_i y_j (x_i x_j + 1)^2 \\ \text{subject to:} \quad & 0 \leq \alpha_i \leq 100, \quad \sum_{i=1}^5 \alpha_i y_i = 0 \end{aligned}$$

Goal

Solve for α_i (support vectors)

Example: SVM with Polynomial Kernel

Solve Dual Problem (QP Solver)

$$\alpha_1=0, \alpha_2=2.5, \alpha_3=0, \alpha_4=7.333, \alpha_5=4.833$$

Identify Support Vectors

Points with $\alpha_i > 0$

Support vectors: $x = \{2, 5, 6\}$

Construct Decision Function

$$\begin{aligned} f(x) &= \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b \\ &= 2.5(1)(2x + 1)^2 + 7.333(-1)(5x + 1)^2 + 4.833(1)(6x + 1)^2 + b \end{aligned}$$

Simplify

$$f(x) = 0.6667x^2 - 5.333x + b$$

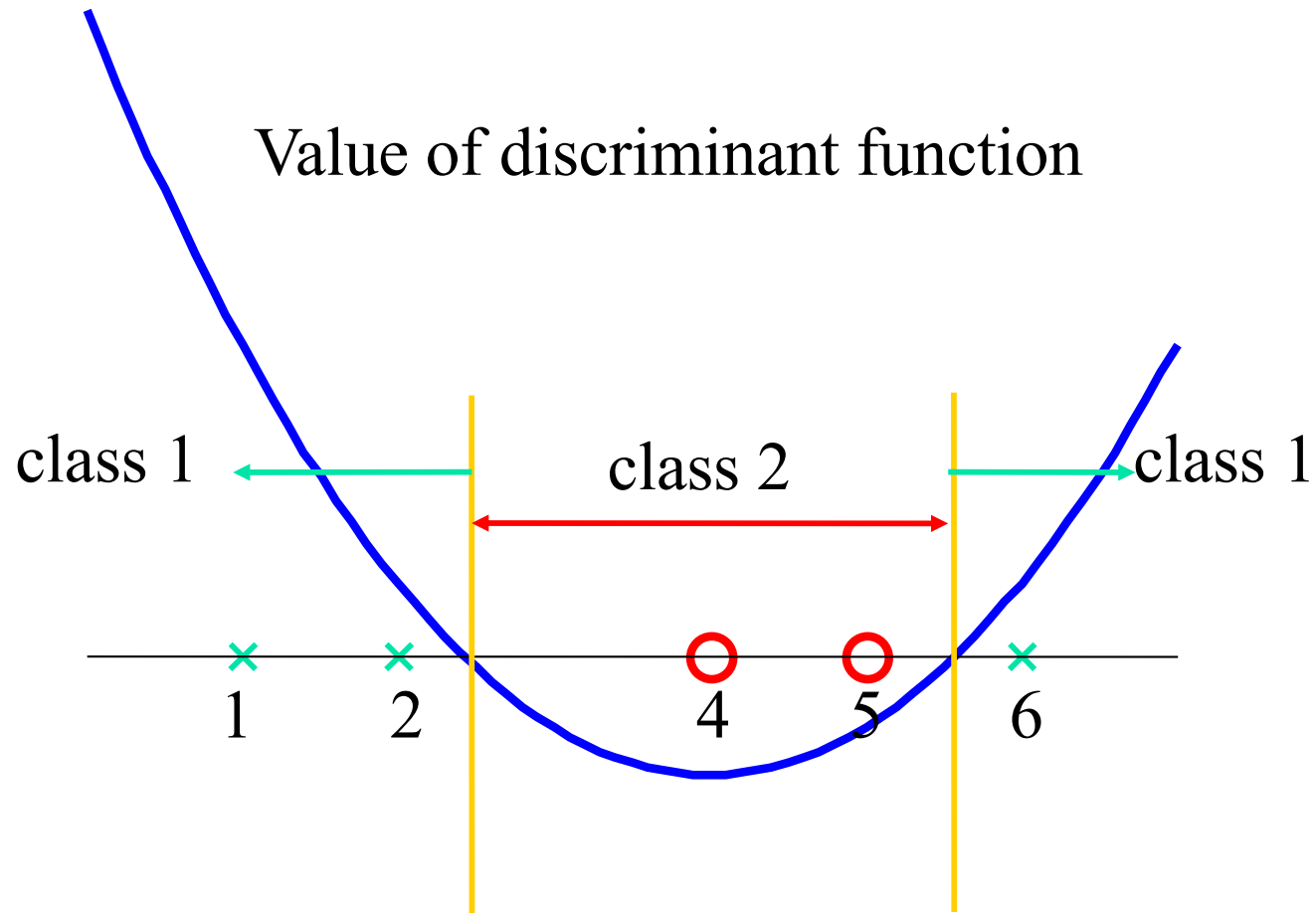
Solve for b

Use any support vector (e.g., $x = 2$): $y_i f(x_i) = 1$ $f(2) = 1 \Rightarrow b = 9$

Final Model

$$f(x) = 0.6667x^2 - 5.333x + 9$$

Example: SVM with Polynomial Kernel



Steps in SVM

1. Prepare data matrix $\{(x_i, y_i)\}$
2. Select a Kernel function
3. Select the error parameter C
4. “Train” the system (to find all α_i)

New data can be classified using α_i and Support Vectors

SVM Strengths

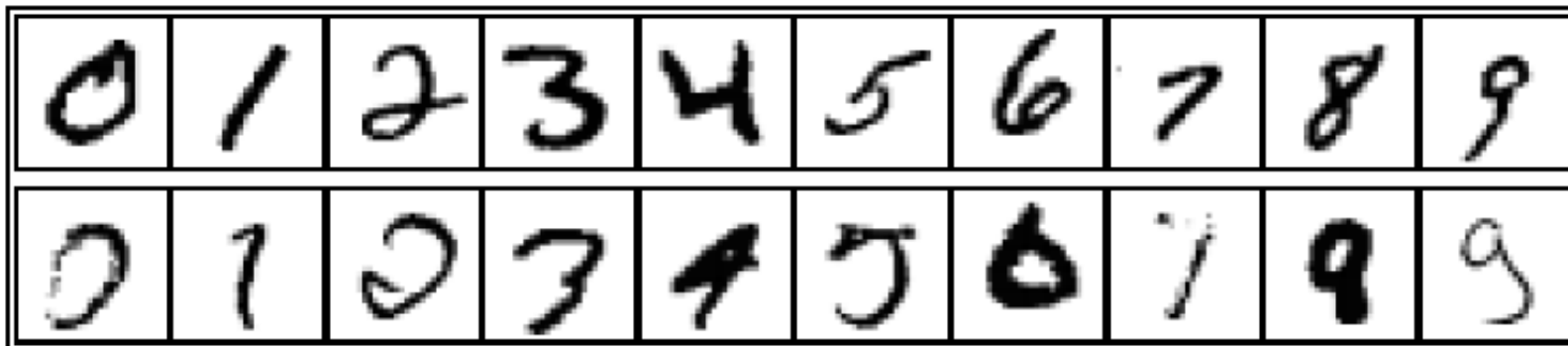
- **Training is relatively easy**
 - Convex optimization → **no local minima**
 - Always find **global optimum**
- **Handles high-dimensional data well**
 - Uses **kernel trick**, only depends on dot products:
 - Implicitly work in high-dimensional space
 - Kernel trick keeps computation manageable
- **Less prone to overfitting**
 - Margin maximization
 - Depends only on support vectors
- **Simple, easy to understand geometric interpretation**
 - Margin + decision boundary

SVM Weakness

- **Computational Cost**
 - Training can be **slow for large datasets**
 - Requires solving a **quadratic programming (QP)** problem
 - Kernel methods scale poorly with number of samples
- **Binary by Design**
 - However, there are tricks to evade this.
- **Sensitive to Noise / Outliers**
 - Support vectors lie near decision boundary
 - Outliers can significantly affect the margin
 - Requires careful tuning of C
- **Kernel Selection & Tuning**
 - Choosing a good kernel is **problem-dependent**
 - No universal rule — often requires experimentation

Applications of SVMs

- Lots of very successful applications!
 - Bioinformatics
 - Machine Vision
 - Text Categorization
 - Ranking (e.g., Google searches)
 - Time series analysis
 - Handwritten Character Recognition



3-nearest-neighbor = 2.4% error

400–300–10 unit MLP = 1.6% error

LeNet: 768–192–30–10 unit MLP = 0.9% error

Current best (kernel machines, vision algorithms) $\approx$ 0.6% error